untangle::executor

Generated by Doxygen 1.18.0

1 executor 1

2 Namespace Index 7
2.1 Namespace List . . . 7

3 Class Index 9
3.1 Class List . . . 9

4 File Index 11
4.1 File List . . . 11

5 Namespace Documentation 13
5.1 untangle Namespace Reference . . . 13
5.1.1 Detailed Description . . . 13

6 Class Documentation 15
6.1 untangle::executor< actionT > Class Template Reference . . . 15
6.1.1 Detailed Description . . . 16
6.1.2 Constructor & Destructor Documentation . . . 16
6.1.2.1 executor() . . . 16
6.1.2.2 ~executor() . . . 17
6.1.3 Member Function Documentation . . . 17
6.1.3.1 wait() . . . 17
6.1.3.2 add_task() . . . 18
6.1.3.3 pending() . . . 18
6.1.3.4 start() . . . 19
6.1.3.5 stop() . . . 19
6.1.4 Member Data Documentation . . . 19
6.1.4.1 on_task_error . . . 19

7 File Documentation 21
7.1 executor.hpp . . . 21

Index 27

Chapter 1

executor

c++ thread pool executor based on async execution

A header-only library: executor.hpp exposes a pool of workers, each one an async execution in continuous mode, fed by a shared queue of tasks.

requirements

C++23, and the async submodule:
git submodule update --init --recursive

async carries its own actuator submodule, which is why the clone has to be recursive. Both are header-only; nothing is built or installed.

The headers report warnings with std::println, so they need a standard library that provides <print> — GCC 14 or newer.

example

```
#include <executor.hpp>
#include <functional>

int main() {
  untangle::executor<std::function<void(void)» pool(4);
  pool.start();

  for (int i = 0; i < 100; ++i) {
    pool.add_task([i] { work(i); }, [] { /* finished */ });
  }

  return 0;  // ~executor finishes what was added, then stops the workers
}
```

the action type

The pool is a template on the callable it runs, the way an execution is — and on the same parameter name, because every task queued is that one callable. One pool runs one signature, and the caller names it:

```
untangle::executor<std::function<int(void)» pool(4);   // each result goes to the task's callback
```

The action type does not have to be a std::function. What an execution requires of it is a nested result↩ _type, which it reads rather than deduces, so any callable declaring one will do — and a pool built on such a callable never touches std::function::result_type, which the standard removed in C++20 and both libc++ and libstdc++ still supply by grace rather than contract.

```
struct my_task {
  using result_type = void;
  void operator()() const;
};

untangle::executor<my_task> pool(4);
```

tasks: work that reports back

Work enters the pool through one door, add_task(), and every task carries the callback it must notify. An action type may take arguments, and add_task() binds them the way its namesake execution::add_task() does — at the door, where the caller still holds them — with the callback after them:

```
untangle::executor<std::function<int(int)» pool(4);
pool.start();

// the callback comes last, after the arguments the task is bound to
pool.add_task([](int n) { return n * 2; }, 21, [](int result) { /* result == 42 */ });
```

What waits for a worker is one callable carrying its own arguments and its callback, so nothing about a task changes between the queue and the worker that runs it. The arguments are copied at add_task() and handed to the task as the pool's own lvalues when it runs: a task is called later, possibly much later, and what the caller passed may be gone by then. A task taking int& therefore mutates the pool's copy, which the caller never sees.

A task that cannot be called with the arguments given is refused by a static_assert in add_task(), which names the mismatch before the errors from inside the binding follow it.

Work that returns void still reports that it finished, with a callback taking nothing — finished is the message and the result is optional:

```
untangle::executor<std::function<void(void)» pool(4);
pool.add_task([] { /* ... */ }, [] { /* finished */ });
```

A task with nothing to bind passes the callback alone, whatever it returns:

```
untangle::executor<std::function<int(void)» pool(4);
pool.add_task([] { return 42; }, [](int result) { /* result == 42 */ });
```

The callback is the last argument, and the signature cannot say so. A parameter pack cannot be followed by a deducible parameter, so it arrives inside the arguments and untangle::bind_task() splits it off. It must be callable with the task's result and return nothing — or callable with nothing at all, when the task returns void. A missing or unusable one is a compile error rather than silence.

Work nobody wants told about still passes a callback, one that ignores what it is given. There is no fire-and-forget door: a void task's callback costs one empty call, and a single door means a single queue, in which the order out is the order in.

```
constexpr auto ignore_result = [](auto&&...) {};   // takes a result, or nothing for a void task
pool.add_task([](int n) { return n * 2; }, 21, ignore_result);
```

A refused task does not notify. add_task() answers false before start(), after stop(), while a wait() is draining, once the destructor has begun, or if the worker it was offered to has stopped — and a refused

task is destroyed rather than queued. The answer is the whole report, so a caller waiting on the callback rather than on the answer would wait for ever. The same is true of work still queued when a stopped pool is destroyed: it never runs, so it never notifies, and the count on stderr is all there is.

Finished does not mean failed. A task that throws is not notified: there is no result to hand over, and for a void task no completion to report either. What it threw goes to on_task_error. And because a callback runs inside the same try as the task that owns it, on_task_error can fire for a task that in fact succeeded — the work was done and only the telling failed.

A callback runs on a worker's thread, inside that worker's drain, and several may be running at once on different workers. It is the same rule on_task_error carries, for the same reason: nothing may escape it.

how work is placed

N executions run in continuous mode behind a shared std::deque of tasks.

Nothing overtakes. A task goes straight to a free worker only when the shared queue is empty; otherwise it joins the back of the queue. Finding a free worker does not let a task jump a queue that already has work in it.

Workers pull rather than being pushed to. When one reports its queue finished — on_finished, raised by execution the moment its list empties — it takes the front of the shared queue. That is what spreads the work, with no scheduling logic of its own: 400 tasks over 4 workers come out 100/100/100/100.

A worker is asked, not tracked. execution::is_busy() is what add_task() reads to find a free worker and what the destructor reads to decide the pool is idle. A tally kept here would be the pool's belief about its workers; this is the workers' own answer, and it cannot drift.

running, stopping, restarting

A pool is built stopped. The constructor makes the workers and wires them up; start() runs them. Until it is called the pool refuses work, because a task taken then would sit unrun and the destructor would wait for it. It is also the moment to assign on_task_error, before anything can throw.

stop() stops the workers, and stops the pool accepting. What a worker is already running it finishes; what is still queued stays queued. stop() does not wait for the workers to leave their threads — only the destructor does that, and that wait is what makes destroying the pool safe.

start() after stop() restarts it, workers and all, and the queue it was holding drains as they pick it up again. Either call is harmless twice over.

wait() is a barrier, not a shutdown. It returns once everything added has run and no worker is busy, and it leaves the pool exactly as it found it — workers still in their threads, nothing destroyed, accepting again. Work added after it runs as it did before, and calling it repeatedly is how you step through work in phases:

```
pool.add_task([] { first(); }, [] {});
pool.wait();                          // first() has run, the pool is still alive
pool.add_task([] { second(); }, [] {});
pool.wait();                          // and so has second()
```

It refuses new work while it waits, then restores accepting to whatever it was. A task that re-adds itself would otherwise keep the drain from ever ending, so add_task() answers false for the duration — and because the previous state is restored rather than assumed, wait() on a stopped pool does not leave it accepting.

So only work accepted before the wait began is what the wait covers. Work offered during it was never queued, and work added after it returns is simply the next batch, for the next wait(). This is a stop-the-world drain, not a transparent one: a producer on another thread sees add_task() answer false while a wait() is in progress, and has to offer the work again afterwards.

A pool that is not running is not waited for. Only a worker takes from the queue, so a stopped pool can never empty it, and a wait insisting otherwise would never return. wait() returns at once instead, leaving what is queued for the next start().

lifetime and failure

The destructor finishes what was added. Its drain is wait(), so a task added before the pool goes out of scope has run by the time it does. What the destructor adds is the shutdown wait() deliberately leaves out, in the order that makes it safe: stop the workers, wait for every one of them to leave its thread, and only then destroy them.

The destructor waits for the pool, not for what the pool's work touched. That distinction is the whole reason wait() is public. The drain guarantees no worker is still running when the pool is destroyed, which is a statement about the pool's own members — a destructor body runs before them, so they are all still alive for it. It says nothing about the lifetime of what a task captured.

For a pool that is a local variable outliving what its work touches, the destructor is enough and nothing else is needed. For a pool held as a member of another class, it is already too late: ~executor() runs during its owner's member destruction, so by then the owner's derived subobject is gone, and so is every member declared after the pool. A callback reaching into either of those runs against destroyed storage however faithfully the pool drains — and on the stack that is not even a fault a sanitizer can name, because the memory is still addressable.

So an owner whose callbacks reach back into itself calls wait() where the whole object graph is still standing — in its own destructor body, ahead of its members, or better still at a point the owner of its peers controls — and the destructor's drain goes back to being a backstop rather than the thing relied upon.

A pool destroyed while stopped is the exception: its queue can never empty, because only a running worker takes from it, so the destructor does not wait for it. Whatever is left is reported on stderr and dropped.

Both waits are unbounded, and both say so while they last. A task that never returns would otherwise leave the destructor silent for ever, so it reports instead — the first wait naming who is still inside a task, the second naming whose thread has not left. The first line below is the drain's, so a plain wait() prints it exactly as a destructor does; the second is the destructor's alone:

```
executor: still waiting to drain after 1s - 1 queued, 2 in a task: pool_1_worker_0, pool_1_worker_1
executor: still waiting to drain after 3s - 0 queued, 1 in a task: pool_1_worker_1
executor: still waiting to stop after 1s - 1 not left its thread: pool_1_worker_1
```

The interval starts at a second and doubles to a ceiling of thirty, so a stuck teardown says something almost at once without a long one becoming a flood. The two waits are kept apart because they answer different questions: a worker can be idle and still be in its thread.

Bounding either wait was never available. Returning early while a worker is still in a task is a use-after-free, and a bounded wait only moves the hang into ~execution(), which spins on its own running state with no timeout. Workers are detached and async::execution has no cancellation, so a pool cannot outlive a task it cannot interrupt. What can be fixed is the silence.

add_task() says whether the task was taken. It returns false before start(), after stop(), while a wait() is draining, and once the destructor has begun. It returns false again if the worker it was offered to has stopped. A refused task is destroyed rather than queued, so a false answer means it will not run.

A task that is still running is refused too, and that is chosen rather than incidental. A pool draining is waiting for exactly that task, so it is tempting to let the work it spawns through — but then a task that re-adds itself could keep the drain from ever ending, and that holds for a wait() just as for a destructor. One rule instead: a pool that is not running takes no work, whoever is asking. add_task() answers false inside the task, so it can tell.

A task that throws does not take the worker with it, and what it threw is not lost. The worker catches it and carries on with the next task; on_task_error is where the throw goes:

```
untangle::executor<std::function<void(void)» pool(4);

pool.on_task_error = [](std::exception_ptr thrown) {
  try {
    std::rethrow_exception(thrown);
  } catch (const std::exception& e) {
    log(e.what());
  }
};
```

It carries a std::exception_ptr because a task may throw something that is not a std::exception, and that is the case most worth hearing about. Assign it before the first add_task() — a worker reads it — and expect it on a worker's thread, with more than one worker possibly inside it at once. Leave it unset and the pool warns on stderr instead, naming the worker.

The name says which pool too. Workers are named <pool>_worker_<i>, after the name the pool was given - executor pool(2, "store_a") has store_a_worker_0 and store_a_worker_1 - or, without one, pool_<n>, numbered across the process: two pools never share a worker name, so every warning - the failure reports above and the destructor's waits alike - says where it came from.

A return value is not collected by the pool: a continuous worker keeps none. It goes to the task's own callback — see tasks: work that reports back above.

building the tests

```
cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build
```

test/CMakeLists.txt fetches googletest at configure time, so the first configure needs network access.

sanitizers

EXECUTOR_SANITIZE builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```
cmake -S test -B test/build-asan -DEXECUTOR_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan
```

Accepted values are address, thread, undefined, or empty. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. address and thread instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

the reference

tools/make_doc.sh builds doc/refman.pdf from the header and this README. It needs doxygen, tectonic and python3, and it fails on any doxygen warning that is not an obsolete-tag notice, so an undocumented member or a broken \ref does not reach the reference.

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle
A thread pool executor built out of untangle::async::execution objects 13

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::executor< actionT >
Runs tasks on a fixed pool of untangle::async::execution workers 15

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

executor.hpp 21

Chapter 5

Namespace Documentation

5.1 untangle Namespace Reference

A thread pool executor built out of untangle::async::execution objects.

Classes

- class executor

  Runs tasks on a fixed pool of untangle::async::execution workers.

Variables

- std::atomic< std::size_t > unnamed_pools = {0}

  Numbers the pools that are not given a name, across every task type, so no two are alike.

5.1.1 Detailed Description

A thread pool executor built out of untangle::async::execution objects.

N workers in continuous mode behind a shared queue. Nothing overtakes what is already queued.

Chapter 6

Class Documentation

6.1 untangle::executor< actionT > Class Template Reference

Runs tasks on a fixed pool of untangle::async::execution workers.

#include <executor.hpp>

Public Member Functions

- executor (std::size_t worker_count, std::string name={})
  Builds worker_count workers. They do not run until start().
- executor (const executor &)=delete
  Not copied or moved: every worker's callbacks hold this pool's this.
- ~executor ()
  Drains what was added, then stops the workers and destroys them.
- void wait ()
  Drains everything added so far, then leaves the pool as it was found.
- template<typename... Args>
  bool add_task (const actionT &task, Args &&... args)
  Gives work that must report to a worker, which queues it.
- template<typename... Args>
  bool add_task (actionT &&task, Args &&... args)
  Gives work that must report to a worker as the overload above does, moving the task into the queue rather than copying it. A temporary and a lambda land here.
- std::size_t pending () const
  How many tasks are waiting in the workers' queues, not yet taken to run.
- void start ()
  Starts every worker, and starts the pool accepting work.
- void stop ()
  Stops every worker. What they are already running, they finish.

Public Attributes

- std::function< void(std::exception_ptr)> on_task_error
  Called with whatever the work threw. Assigned by the caller.

6.1.1 Detailed Description

template<typename actionT>
class untangle::executor< actionT >

Runs tasks on a fixed pool of untangle::async::execution workers.

add_task() is the one door: it carries a callback and reports the result to it.

Each worker keeps its own queue. A task goes to a worker when it is submitted - the first idle one, else the one given the fewest since it last drained - and waits there for that worker.

Template Parameters

actionT	- The callable the pool runs. One pool runs one signature, arguments and result.

6.1.2 Constructor & Destructor Documentation

6.1.2.1 executor()

template<typename actionT>
untangle::executor< actionT >::executor (
    std::size_t worker_count,
    std::string name = {}) [inline], [explicit]

Builds worker_count workers. They do not run until start().

Parameters

worker_count	- How many workers to run. Must not be zero.
name	- What its workers are named after, <name>_worker_<i>, in every warning they print. Left empty, it is pool_<n>, numbered across the process, so two pools are never alike.

Exceptions

std::invalid_argument	- worker_count is zero: such a pool could never run its work.

6.1.2.2 ~executor()

template<typename actionT>
untangle::executor< actionT >::~executor () [inline]

Drains what was added, then stops the workers and destroys them.

The drain is wait(); the shutdown after it stops the workers, waits for each to leave its thread, and destroys them only then. Both waits are unbounded and report on stderr meanwhile.

Remarks

Nothing is dropped: a worker runs what it was given before it leaves its thread, a pool stopped earlier included.

Attention

This waits for the pool, not for what its work touched: see wait().

6.1.3 Member Function Documentation

6.1.3.1 wait()

template<typename actionT>
void untangle::executor< actionT >::wait () [inline]

Drains everything added so far, then leaves the pool as it was found.

Drained means no worker has anything queued or running, so every task accepted before this has run, and so has the callback of one - a callback runs inside the worker's drain, not after it.

A barrier, not a shutdown: the workers keep their threads and work added afterwards runs as before. Reusable, and one at a time - a second caller drains once the first has returned.

The wait is unbounded and reports on stderr while it lasts.

Attention

New work is refused meanwhile, then accepting is restored, so a task that re-adds itself cannot keep the drain from ending. Only work accepted before it began counts.

Remarks

A stopped pool is waited for too: its workers run what they were given before they leave, so the wait ends.

Attention

It waits for the work, not for what the work touched. Held as a member, ~executor() runs once the owner's derived parts and later members are gone: call this before that.

6.1.3.2 add_task()

```
template<typename actionT>
template<typename... Args>
bool untangle::executor< actionT >::add_task (
                const actionT & task,
                Args &&... args)   [inline]
```

Gives work that must report to a worker, which queues it.

The first idle worker takes it, the warmest one; otherwise the worker given the fewest since it last drained, and the task waits behind that worker's queue - in the order it arrived there. Once the task has run, its callback is invoked with the result, on the worker's thread.

Attention

The last argument is the callback, which the signature cannot say: it arrives in args and untangle::↩ bind_task() splits it off. A missing or unusable one is a static_assert, and so is a task that cannot be called with the arguments before it.

A task that throws is not notified - what it threw goes to on_task_error - and a refused one does not notify either, so a caller waiting on the callback alone waits for ever.

An empty callback or action is refused here too, silently: the answer is the report. Only a run time check can see it, and a task that can neither run nor notify is turned away before any worker is asked.

Template Parameters

Args	- The argument types to bind, of which the last is the callback.

Parameters

task	- The task to run, copied into the worker's queue.
args	- What to bind, followed by the callback. Copied by the worker's queue, so a task taking a reference gets the pool's copy.

Returns

true - accepted, and the callback will be notified once it has run.

false - refused by a pool not started, stopped, draining or being destroyed, by a stopped worker, or because the task could never do its job. It will not run or notify.

6.1.3.3 pending()

```
template<typename actionT>
std::size_t untangle::executor< actionT >::pending () const   [inline]
```

How many tasks are waiting in the workers' queues, not yet taken to run.

Remarks

Advisory: true the moment it is read.

6.1.3.4 start()

template<typename actionT>
void untangle::executor< actionT >::start () [inline]

Starts every worker, and starts the pool accepting work.

A pool refuses work until this is called, and again after stop() until it is called anew, which revives the workers too. Harmless twice over. wait() needs nothing from here.

6.1.3.5 stop()

template<typename actionT>
void untangle::executor< actionT >::stop () [inline]

Stops every worker. What they are already running, they finish.

What the workers were given they still run, then leave; anything added afterwards is refused, a running task included. start() revives the workers; harmless twice over.

Attention

It does not wait for the workers to leave their threads - only the destructor does, and that wait is what makes destroying the pool safe.

6.1.4 Member Data Documentation

6.1.4.1 on_task_error

template<typename actionT>
std::function<void(std::exception_ptr)> untangle::executor< actionT >::on_task_error

Called with whatever the work threw. Assigned by the caller.

The only report of failed work, since add_task() answered long before it ran. Left unset, the pool warns on stderr. Rethrow the exception_ptr to read it.

Remarks

A task's failure arrives here, and so does its callback's - so this can fire for a task that succeeded, when only the telling failed.

Attention

It runs on a worker's thread, several at once. Assign it before the first add_task(), and let nothing escape it or a callback.

The documentation for this class was generated from the following file:

- executor.hpp

Chapter 7

File Documentation

7.1 executor.hpp

```
00001 // Copyright (c) 2026 Nicolae Popescu. MIT License.
00002
00008 #ifndef UNTANGLE_EXECUTOR_EXECUTOR_HPP
00009 #define UNTANGLE_EXECUTOR_EXECUTOR_HPP
00010
00011 #include <algorithm>
00012 #include <async.hpp>
00013 #include <atomic>
00014 #include <chrono>
00015 #include <condition_variable>
00016 #include <cstddef>
00017 #include <exception>
00018 #include <functional>
00019 #include <memory>
00020 #include <mutex>
00021 #include <print>
00022 #include <stdexcept>
00023 #include <string>
00024 #include <thread>
00025 #include <tuple>
00026 #include <type_traits>
00027 #include <utility>
00028 #include <vector>
00029
00030 namespace untangle {
00031
00033 inline std::atomic<std::size_t> unnamed_pools = {0};
00034
00045 template <typename actionT>
00046 class executor {
00047  public:
00056   explicit executor(std::size_t worker_count, std::string name = {}) {
00057     if (worker_count == 0) {
00058       throw std::invalid_argument("executor: worker_count must not be zero");
00059     }
00060
00061     if (name.empty()) {
00062       name = "pool_" + std::to_string(unnamed_pools.fetch_add(1) + 1);
00063     }
00064
00065     workers_.reserve(worker_count);
00066     given_ = std::make_unique<given_count[]>(worker_count);
00067
00068     for (std::size_t index = 0; index < worker_count; ++index) {
00069       auto next = worker_execution::create_instance(name + "_worker_" + std::to_string(index));
00070
00071       // Tells the pool this worker has run everything it was given.
00072       next->on_finished = [this, index] { worker_drained(index); };
00073
00074       // Carries what a task throws to the caller. Assigned here because a worker thread reads it.
00075       next->on_error = [this, worker = next->name](std::exception_ptr thrown) {
00076         report_task_error(worker, thrown);
00077       };
00078
00079       // Adds the worker to the poll the destructor waits on.
00080       poll_.add(*next);
00081
```

```
00082       workers_.push_back(std::move(next));
00083     }
00084   }
00085
00087   executor(const executor&) = delete;
00088   executor& operator=(const executor&) = delete;
00089   executor(executor&&) = delete;
00090   executor& operator=(executor&&) = delete;
00091
00103   ~executor() {
00104     // Nothing is stopped before this, or the queue could never empty. wait() leaves the pool
00105     // accepting, so stop() follows at once: that window is the only one a shutdown would take.
00106     wait();
00107     stop();
00108
00109     // Waits for every worker to leave its thread, which is what makes destroying them safe. A
00110     // different question from the wait above: a worker can be idle and still be in its thread.
00111     // A worker leaves within microseconds of stop(), so the first tick is short and doubles up to
00112     // the ceiling: a fixed tick made every destruction sleep it once.
00113     auto interval = std::chrono::milliseconds(report_first_ms);
00114     auto waited = std::chrono::microseconds(0);
00115     auto tick = std::chrono::microseconds(poll_first_us);
00116
00117     while (poll_.is_running()) {  // time of check
00118       std::this_thread::sleep_for(tick);
00119       waited += tick;
00120       tick = std::min(tick * 2, std::chrono::microseconds(poll_max_ms * 1000));
00121
00122       if (waited >= interval) {
00123         std::println(stderr, "executor: still waiting to stop after {}s - {} not left its thread{}",
00124                      std::chrono::duration_cast<std::chrono::seconds>(waited).count(),
00125                      count_workers(&worker_execution::is_running),
00126                      name_workers(&worker_execution::is_running));
00127
00128         interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00129       }
00130     }
00131
00132     // time of use: no worker thread is left to reach workers_.
00133     workers_.clear();
00134   }
00135
00156   void wait() {
00157     // One drain at a time: two threads between the exchange below and the restore at the end would
00158     // read each other's cleared flag, and the last one out could leave the pool refusing work.
00159     std::lock_guard<std::mutex> drain(wait_mutex_);
00160
00161     // Put back exactly as found below: a wait must not leave a stopped pool accepting.
00162     const bool was_running = running_.exchange(false);
00163
00164     {
00165       std::unique_lock<std::mutex> lock(mutex_);
00166
00167       // Counted under the lock, before the first check: a worker that drains after this sees it and
00168       // notifies, and one that drained before it is seen by that first check.
00169       ++waiting_;
00170
00171       // Waits for every worker to go idle.
00172       auto interval = std::chrono::milliseconds(report_first_ms);
00173       auto waited = std::chrono::milliseconds(0);
00174
00175       while (!finished_cv_.wait_for(lock, interval, [this] { return nothing_running(); })) {
00176         waited += interval;
00177
00178         std::println(
00179             stderr, "executor: still waiting to drain after {}s - {} queued, {} in a task{}",
00180             waited.count() / 1000, total_pending(), count_workers(&worker_execution::is_busy),
00181             name_workers(&worker_execution::is_busy));
00182
00183         interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00184       }
00185
00186       --waiting_;
00187     }
00188
00189     running_ = was_running;
00190   }
00191
00219   template <typename... Args>
00220   bool add_task(const actionT& task, Args&&... args) {
00221     // The call the pool will make, which is not quite the one written at the call site: what
00222     // reaches the task is the copies the worker binds, so that is the call that has to compile.
00223     // Without this the mismatch surfaces deep inside bind_task()'s std::apply.
00224     static_assert(invocable_with_bound<Args...>(),
00225                   "executor: the task cannot be called with the arguments given to add_task() "
00226                   "before its callback");
00227
```

```
00228     if (!running_ || task_is_empty(task, args...)) {
00229       return false;
00230     }
00231
00232     return give_to_worker(pick_worker(), task, std::forward<Args>(args)...);
00233   }
00234
00239   template <typename... Args>
00240   bool add_task(actionT&& task, Args&&... args) {
00241     static_assert(invocable_with_bound<Args...>(),
00242                   "executor: the task cannot be called with the arguments given to add_task() "
00243                   "before its callback");
00244
00245     if (!running_ || task_is_empty(task, args...)) {
00246       return false;
00247     }
00248
00249     return give_to_worker(pick_worker(), std::move(task), std::forward<Args>(args)...);
00250   }
00251
00257   std::size_t pending() const { return total_pending(); }
00258
00265   void start() {
00266     // The workers first, so anything accepted after this is handed to one that is running.
00267     for (auto& one : workers_) {
00268       one->start();
00269     }
00270
00271     running_ = true;
00272   }
00273
00284   void stop() {
00285     // First, so that nothing is accepted for workers that are about to stop. A task already on a
00286     // worker still runs; one arriving from here is refused.
00287     running_ = false;
00288
00289     // No lock: a worker draining after stop() runs on_finished, which may take mutex_.
00290     for (auto& one : workers_) {
00291       one->stop();
00292     }
00293   }
00294
00307   std::function<void(std::exception_ptr)> on_task_error;
00308
00309  private:
00316   void report_task_error(const std::string& worker, std::exception_ptr thrown) {
00317     if (on_task_error) {
00318       on_task_error(thrown);
00319       return;
00320     }
00321
00322     try {
00323       std::rethrow_exception(thrown);
00324     } catch (const std::exception& e) {
00325       std::println(stderr, "warning: executor task on '{}' threw: {}", worker, e.what());
00326     } catch (...) {
00327       std::println(stderr, "warning: executor task on '{}' threw an unknown type", worker);
00328     }
00329   }
00330
00340   template <typename... Args>
00341   static consteval bool invocable_with_bound() {
00342     if constexpr (sizeof...(Args) == 0) {
00343       return true;
00344     } else {
00345       return invocable_with<std::tuple<std::decay_t<Args>...>(
00346           std::make_index_sequence<sizeof...(Args) - 1>{});
00347     }
00348   }
00349
00351   template <typename tupleT, std::size_t... index>
00352   static consteval bool invocable_with(std::index_sequence<index...>) {
00353     return std::is_invocable_v<actionT, std::tuple_element_t<index, tupleT>&...>;
00354   }
00355
00356   using worker_execution = untangle::async::execution<actionT>;
00357
00360   struct alignas(128) given_count : std::atomic<std::size_t> {};
00361
00367   std::size_t pick_worker() const {
00368     std::size_t best = 0;
00369     std::size_t best_count = given_[0].load(std::memory_order_relaxed);
00370
00371     for (std::size_t index = 0; index < workers_.size(); ++index) {
00372       const std::size_t count = given_[index].load(std::memory_order_relaxed);
00373       if (count == 0) {
00374         return index;
```

```
00375       }
00376       if (count < best_count) {
00377         best = index;
00378         best_count = count;
00379       }
00380     }
00381
00382     return best;
00383   }
00384
00394   template <typename action_t, typename... Args>
00395   bool give_to_worker(std::size_t index, action_t&& task, Args&&... args) {
00396     // No lock: the counts only steer pick_worker(), so a stale one costs balance, not correctness.
00397     given_[index].fetch_add(1, std::memory_order_relaxed);
00398
00399     if (workers_[index]->add_task(std::forward<action_t>(task), std::forward<Args>(args)...)) {
00400       return true;
00401     }
00402
00403     // Only a stopped worker refuses: add_task() turned an empty task away before. Not counted, or
00404     // an idle worker would look busy until it is next given work and drains.
00405     forget_given(index);
00406     return false;
00407   }
00408
00414   template <typename... Args>
00415   static bool task_is_empty(const actionT& task, const Args&... args) {
00416     if constexpr (testable_for_emptiness<actionT>) {
00417       if (!task) {
00418         return true;
00419       }
00420     }
00421
00422     if constexpr (sizeof...(Args) > 0) {
00423       const auto& callback = std::get<sizeof...(Args) - 1>(std::forward_as_tuple(args...));
00424       if constexpr (testable_for_emptiness<std::decay_t<decltype(callback)>>) {
00425         if (!callback) {
00426           return true;
00427         }
00428       }
00429     }
00430
00431     return false;
00432   }
00433
00437   void forget_given(std::size_t index) {
00438     std::size_t count = given_[index].load(std::memory_order_relaxed);
00439     while (count > 0 &&
00440            !given_[index].compare_exchange_weak(count, count - 1, std::memory_order_relaxed)) {
00441     }
00442   }
00443
00447   void worker_drained(std::size_t index) {
00448     given_[index].store(0, std::memory_order_relaxed);
00449
00450     // The lock only for a wait(), which counts itself under it before its first check.
00451     if (waiting_.load() > 0) {
00452       std::lock_guard<std::mutex> lock(mutex_);
00453       finished_cv_.notify_all();
00454     }
00455   }
00456
00458   std::size_t total_pending() const {
00459     std::size_t total = 0;
00460     for (const auto& one : workers_) {
00461       total += one->pending();
00462     }
00463     return total;
00464   }
00465
00471   bool nothing_running() const {
00472     for (const auto& one : workers_) {
00473       if (one->is_busy()) {
00474         return false;
00475       }
00476     }
00477
00478     return true;
00479   }
00480
00486   std::size_t count_workers(bool (worker_execution::*state)() const) const {
00487     std::size_t count = 0;
00488
00489     for (const auto& one : workers_) {
00490       if ((one.get()->*state)()) {
00491         ++count;
00492       }
```

```
00493     }
00494
00495     return count;
00496   }
00497
00504   std::string name_workers(bool (worker_execution::*state)() const) const {
00505     std::string names;
00506
00507     for (const auto& one : workers_) {
00508       if ((one.get()->*state)()) {
00509         names += names.empty() ? ": " : ", ";
00510         names += one->name;
00511       }
00512     }
00513
00514     return names;
00515   }
00516
00519   static constexpr int poll_first_us = 100;
00520   static constexpr int poll_max_ms = 50;
00521
00523   static constexpr int report_first_ms = 1000;
00524   static constexpr int report_max_ms = 30000;
00525
00526   mutable std::mutex mutex_;
00527   std::condition_variable finished_cv_;
00528
00530   std::mutex wait_mutex_;
00531
00534   async::execution_poll poll_;
00535
00536   std::vector<std::shared_ptr<worker_execution» workers_;
00537
00539   std::unique_ptr<given_count[]> given_;
00540
00542   std::atomic<std::size_t> waiting_ = {0};
00545   std::atomic_bool running_ = {false};
00546 };
00547
00548 }  // namespace untangle
00549
00550 #endif  // UNTANGLE_EXECUTOR_EXECUTOR_HPP
```

Index

~executor
untangle::executor< actionT >, 16

add_task
untangle::executor< actionT >, 17

executor, 1
untangle::executor< actionT >, 16

on_task_error
untangle::executor< actionT >, 19

pending
untangle::executor< actionT >, 18

start
untangle::executor< actionT >, 18
stop
untangle::executor< actionT >, 19

untangle, 13
untangle::executor< actionT >, 15
~executor, 16
add_task, 17
executor, 16
on_task_error, 19
pending, 18
start, 18
stop, 19
wait, 17

wait
untangle::executor< actionT >, 17